

EXPERIMENT 8

Student Name: ARNAV SHARMA

Branch: AIT-CSE-AIML

Semester: 5

Subject Name: ADBMS

UID: 24BAI70781

Section/Group: 24AIT_KRG1_B

Subject Code: 24CSP-310

Question 1:

A company maintains customer information, product information, and sales order information in three separate tables: `customer_master`, `product_catalog`, and `sales_orders`.

The company wants to provide an order report to an external client. The report must contain the **Order ID, Order Date, Customer Name, Product Name, and Final Amount**.

For every order, first calculate the **Gross Amount** using:

Gross Amount = Unit Price × Quantity

Then calculate the **Discount Amount** using:

Discount Amount = Gross Amount × Discount Percentage / 100

Finally, calculate the **Final Amount** using:

Final Amount = Gross Amount – Discount Amount

Create a **complex view** using the three tables to generate this report. The view should expose only the required five columns and should not expose the customer's phone/email, product brand/unit price, or the order's quantity/discount percentage.

The company also wants to provide access to an external client. Create a PostgreSQL user named `client_user` and give this user **only SELECT permission on the view**. The client must not have direct access to the three original tables and must not be allowed to modify the view data.

Solution:

```
CREATE TABLE customer_master
```

```
(  
    customer_id VARCHAR(5) PRIMARY KEY,  
    full_name VARCHAR(50) NOT NULL,
```

EXPERIMENT 8

city VARCHAR(30)

);

CREATE TABLE product_catalog

(

product_id VARCHAR(5) PRIMARY KEY,

product_name VARCHAR(50) NOT NULL,

unit_price NUMERIC(10,2) NOT NULL

);

CREATE TABLE sales_orders

(

order_id SERIAL PRIMARY KEY,

product_id VARCHAR(5)

REFERENCES product_catalog(product_id),

quantity INT NOT NULL,

customer_id VARCHAR(5)

REFERENCES customer_master(customer_id),

discount_percent NUMERIC(5,2),

order_date DATE NOT NULL

);

INSERT INTO customer_master

(customer_id, full_name, city)

VALUES

EXPERIMENT 8

('C1', 'Amit Sharma', 'Delhi'),
('C2', 'Priya Verma', 'Mumbai'),
('C3', 'Ravi Kumar', 'Bangalore'),
('C4', 'Neha Singh', 'Kolkata'),
('C5', 'Arjun Mehta', 'Hyderabad');

```
INSERT INTO product_catalog
(product_id, product_name, unit_price)
VALUES
('P1', 'Smartphone X100', 25000),
('P2', 'Laptop Pro 15', 65000),
('P3', 'Wireless Earbuds', 5000),
('P4', 'Smartwatch Fit', 30000),
('P5', 'Gaming Console', 45000);
```

```
INSERT INTO sales_orders
(product_id, quantity, customer_id, discount_percent, order_date)
VALUES
('P1', 2, 'C1', 5, '2025-09-01'),
('P2', 1, 'C2', 10, '2025-09-02'),
('P3', 3, 'C3', 0, '2025-09-03'),
('P4', 1, 'C4', 8, '2025-09-04'),
('P5', 1, 'C5', 12, '2025-09-05'),
('P1', 1, 'C2', 5, '2025-09-06'),
('P3', 2, 'C1', 0, '2025-09-07');
```

--explain analyze

EXPERIMENT 8

create or replace view vW_FINAL_AMOUNT_CALCULATION

as

select o.order_id,o.order_date,c.full_name,p.product_name,

((p.unit_price*o.quantity)-((p.unit_price*o.quantity)*o.discount_percent/100)) as final_amount

from sales_orders as o

join customer_master as c on o.customer_id = c.customer_id

join product_catalog as p on p.product_id = o.product_id

select * from vW_FINAL_AMOUNT_CALCULATION;

create user CLIENT_123

PASSWORD '12345';

grant select on vW_FINAL_AMOUNT_CALCULATION to CLIENT_123

Output:

	order_id integer	order_date date	full_name character varying (50)	product_name character varying (50)	final_amount numeric
1	1	2025-09-01	Amit Sharma	Smartphone X100	47500.000000000000000000
2	2	2025-09-02	Priya Verma	Laptop Pro 15	58500.000000000000000000
3	3	2025-09-03	Ravi Kumar	Wireless Earbuds	15000.000000000000000000
4	4	2025-09-04	Neha Singh	Smartwatch Fit	27600.000000000000000000
5	5	2025-09-05	Arjun Mehta	Gaming Console	39600.000000000000000000
6	6	2025-09-06	Priya Verma	Smartphone X100	23750.000000000000000000
7	7	2025-09-07	Amit Sharma	Wireless Earbuds	10000.000000000000000000

EXPERIMENT 8

Question 2:

An e-commerce company wants to regularly analyze the performance of its **product categories**.

The company stores product, category, customer, and order information in separate tables. To generate the category performance report, the database needs to perform multiple **JOINS, GROUP BY, aggregate functions, DISTINCT counting, calculated values, and CASE statements**.

Since the same report is generated frequently, executing the complete analytical query every time can increase database workload.

Create a PostgreSQL **Materialized View** named: mv_category_sales

Also demonstrate that when new order data is inserted, the Materialized View does **not automatically reflect the change** and must be refreshed.

Finally, use EXPLAIN ANALYZE to compare the performance of the original analytical query with the Materialized View query.

The report must contain:

- Category ID
- Category Name
- Total Products Sold
- Total Orders
- Unique Customers
- Total Revenue
- Average Revenue per Order
- Category Performance

Solution:

```
CREATE TABLE category_master (  
    category_id VARCHAR(5) PRIMARY KEY,  
    category_name VARCHAR(50) NOT NULL  
);
```

EXPERIMENT 8

```
CREATE TABLE product_master (  
    product_id VARCHAR(5) PRIMARY KEY,  
    product_name VARCHAR(50) NOT NULL,  
    category_id VARCHAR(5)  
        REFERENCES category_master(category_id)  
);
```

```
CREATE TABLE customer_master (  
    customer_id VARCHAR(5) PRIMARY KEY,  
    customer_name VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE order_details (  
    order_id VARCHAR(5) NOT NULL,  
    customer_id VARCHAR(5)  
        REFERENCES customer_master(customer_id),  
    product_id VARCHAR(5)  
        REFERENCES product_master(product_id),  
    quantity INT NOT NULL,  
    unit_price NUMERIC(10,2) NOT NULL  
);
```

```
INSERT INTO category_master  
(category_id, category_name)  
VALUES  
( 'C1', 'Electronics'),  
( 'C2', 'Clothing'),  
( 'C3', 'Books');
```

EXPERIMENT 8

INSERT INTO product_master

(product_id, product_name, category_id)

VALUES

('P101', 'Laptop', 'C1'),

('P102', 'Headphones', 'C1'),

('P201', 'Jacket', 'C2'),

('P202', 'Shoes', 'C2'),

('P301', 'SQL Book', 'C3');

INSERT INTO customer_master

(customer_id, customer_name)

VALUES

('U1', 'Aman'),

('U2', 'Priya'),

('U3', 'Rahul'),

('U4', 'Simran');

INSERT INTO order_details

(order_id, customer_id, product_id, quantity, unit_price)

VALUES

('O101', 'U1', 'P101', 1, 5000),

('O102', 'U2', 'P102', 2, 1000),

('O103', 'U1', 'P201', 1, 3000),

('O104', 'U3', 'P202', 2, 2000),

('O105', 'U4', 'P301', 3, 500),

('O106', 'U2', 'P101', 1, 5000),

('O107', 'U3', 'P201', 1, 3000),

('O108', 'U4', 'P102', 1, 1000);

EXPERIMENT 8

CREATE MATERIALIZED VIEW MV_CATEGORY_SALE

AS

SELECT

c.category_id,

c.category_name,

SUM(od.quantity) AS total_products_sold,

COUNT(DISTINCT od.order_id) AS total_orders,

COUNT(DISTINCT od.customer_id) AS unique_customers,

SUM(od.quantity * od.unit_price) AS total_revenue,

ROUND(

SUM(od.quantity * od.unit_price)

/ COUNT(DISTINCT od.order_id),

2

) AS avg_revenue_per_order,

CASE

WHEN SUM(od.quantity * od.unit_price) >= 10000

THEN 'Excellent'

WHEN SUM(od.quantity * od.unit_price) >= 6000

THEN 'Good'

WHEN SUM(od.quantity * od.unit_price) >= 3000

THEN 'Average'

EXPERIMENT 8

ELSE 'Low'

END AS category_performance

FROM category_master c

JOIN product_master p

ON c.category_id = p.category_id

JOIN order_details od

ON p.product_id = od.product_id

GROUP BY

c.category_id,

c.category_name;

EXPLAIN ANALYZE

SELECT * FROM MV_CATEGORY_SALE;

--REFRESH MV

REFRESH MATERIALIZED VIEW MV_CATEGORY_SALE;

Output:

Data Output Messages Notifications								
	category_id character varying (5)	category_name character varying (50)	total_products_sold bigint	total_orders bigint	unique_customers bigint	total_revenue numeric	avg_revenue_per_order numeric	category_perfo text
1	C1	Electronics	5	4	3	13000.00	3250.00	Excellent
2	C2	Clothing	4	3	2	10000.00	3333.33	Excellent
3	C3	Books	3	1	1	1500.00	1500.00	Low